**CENTRO UNIVERSITÁRIO UNIFECAF**
**CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS**
Disciplina: Web Development Front-End
Aluno: Emerson Marques Cardoso dos Santos | RA: 92924

# CONSUMO DE API COM JAVASCRIPT

Galeria de personagens com a Rick and Morty API
2026

## 1 INTRODUÇÃO

Este trabalho reúne a base teórica do projeto prático da disciplina de Web Development Front-End: uma página web feita somente com HTML, CSS e JavaScript que consome dados reais da Rick and Morty API e mostra os personagens em cards criados dinamicamente. A seguir, explico como uma API funciona, o que é a manipulação do DOM, quais funções do JavaScript foram usadas, por que escolhi essa API e quais são as regras para utilizá-la.

## 2 API E CONSUMO DE DADOS EM TEMPO REAL

API, sigla de *Application Programming Interface*, é um conjunto de regras que permite que um programa converse com outro. Na web, as mais comuns seguem o modelo REST: o servidor disponibiliza endereços, chamados de *endpoints*, e, quando o navegador envia uma requisição HTTP do tipo GET para um deles, recebe como resposta dados em formato JSON. Nessa relação, o navegador é o cliente e o servidor da API é quem guarda as informações.

Quando se fala em dados “em tempo real”, quer dizer que a informação não está escrita dentro do HTML. A cada acesso ou clique, o JavaScript pede os dados ao servidor, espera a resposta de forma assíncrona, sem travar a página, e usa o resultado para montar a tela. Assim, se a API for atualizada, o site passa a mostrar o dado novo sem que o código precise ser alterado.

## 3 DOM E CRIAÇÃO DE ELEMENTOS DINÂMICOS

O DOM (*Document Object Model*) é a representação do documento HTML como uma árvore de objetos, que o JavaScript consegue ler e modificar depois que a página já carregou. Manipular o DOM, portanto, é criar, alterar ou remover elementos por meio de código. No projeto, o HTML tem apenas um contêiner vazio (`<section id="lista-cards">`). Cada card, com título, imagem e textos, é construído pelo JavaScript a partir dos dados recebidos, o que evita repetir código e permite exibir qualquer quantidade de personagens com a mesma estrutura.

## 4 FUNÇÕES UTILIZADAS

O consumo da API depende de poucas funções. O `fetch()` envia a requisição e devolve uma *Promise*, objeto que representa um resultado que ainda vai chegar. O `then()` define o que fazer quando ele chega: primeiro o método `json()` converte o corpo da resposta em objeto JavaScript e, no `then()` seguinte, os dados são usados. O `catch()` trata as falhas, como falta de internet ou busca sem resultado. Com os dados em mãos, o `document.createElement()` cria cada elemento na memória, propriedades como `textContent` e `src` o preenchem e o `appendChild()` o coloca na página. Por fim, o `addEventListener()` faz o site reagir a cliques e à digitação. O Quadro 1 resume o trecho central do projeto.

Quadro 1 – Consumo da API e criação dinâmica de um card

```
fetch("https://rickandmortyapi.com/api/character?page=1")
  .then(resposta => resposta.json())
  .then(dados => {
    dados.results.forEach(p => {
      const card = document.createElement("article");
      const nome = document.createElement("h2");
      nome.textContent = p.name;
      const img = document.createElement("img");
      img.src = p.image;
      card.append(nome, img);
      document.getElementById("lista-cards").appendChild(card);
    });
  })
  .catch(erro => console.error(erro));
```

Fonte: elaborado pelo autor (2026).

## 5 MOTIVO DA ESCOLHA DA API

Escolhi a Rick and Morty API por ser gratuita, não exigir cadastro nem token e aceitar requisições feitas diretamente do navegador, o que deixa o projeto simples de executar e de demonstrar. Ela também devolve imagem, status, espécie, gênero, origem e localização de cada personagem, informações que se encaixam bem no layout do card pedido no trabalho. Por fim, a série é muito conhecida e a API tem centenas de personagens, o que justifica recursos como paginação, busca e filtro.

## 6 REGRAS DE ACESSO E DOCUMENTAÇÃO

A documentação oficial está em https://rickandmortyapi.com/documentation e a URL base é https://rickandmortyapi.com/api. A API é somente de leitura (GET) e não exige autenticação. Segundo a documentação, existe um limite de requisições por IP, de cerca de 10.000 por dia, valor mais que suficiente para este projeto. Foram usados os parâmetros `page`, `name` e `status`, além da consulta por vários ids de uma vez (`/character/1,2,3`), usada nos favoritos.

A resposta é um objeto JSON com `info` (total de itens e de páginas) e `results`, a lista de personagens, cada um com `id`, `name`, `status`, `species`, `gender`, `origin`, `location`, `image` e `episode`. Quando a busca não encontra ninguém, a API responde com erro 404, tratado no código para mostrar uma mensagem amigável.

## 7 CONSIDERAÇÕES FINAIS

O projeto mostra, na prática, o ciclo completo de um front-end moderno: pedir dados reais a uma API, tratar a resposta e os erros e transformar o JSON em elementos criados dinamicamente no DOM, usando apenas HTML, CSS e JavaScript.

## REFERÊNCIAS

MDN WEB DOCS. **Usando a Fetch API**. [*S. l.*]: Mozilla, [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Fetch_API/Using_Fetch. Acesso em: 4 nov. 2026.

MDN WEB DOCS. **Document: método createElement()**. [*S. l.*]: Mozilla, [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Document/createElement. Acesso em: 4 nov. 2026.

MDN WEB DOCS. **Node: método appendChild()**. [*S. l.*]: Mozilla, [s. d.]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Node/appendChild. Acesso em: 4 nov. 2026.

RICK AND MORTY API. **Documentation**. [*S. l.*]: Rick and Morty API, [s. d.]. Disponível em: https://rickandmortyapi.com/documentation. Acesso em: 4 nov. 2026.